De la maquette au modèle

Notre cas d'étude, expliqué pas à pas

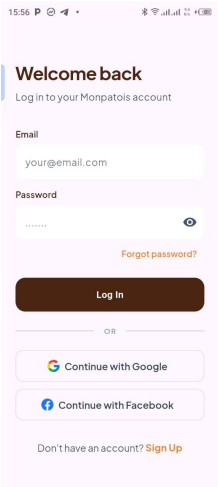
Note de synthèse — Recherche Master II

19 juillet 2026

Plan

1. Notre cas d'étude
2. La maquette haute fidélité
3. Le modèle
4. Détection sur notre maquette
5. Le lien entre maquette et modèle

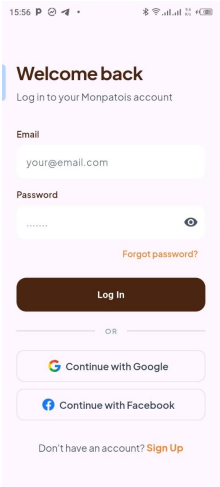
Le cas d'étude que nous allons suivre



Objectif final

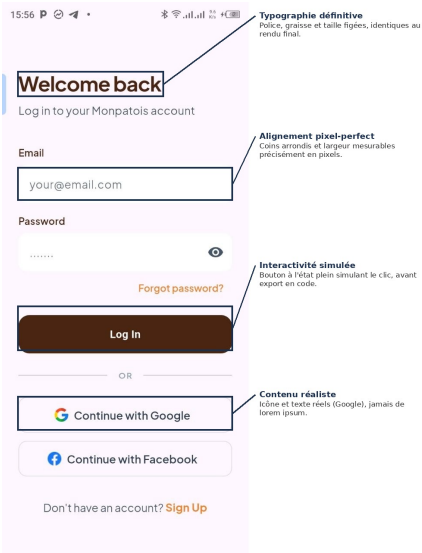
Comprendre comment cette image se transforme en un **modèle structuré** (graphe puis arbre), avant génération de code.

Qu'est-ce qu'une maquette haute fidélité ?



Ce que vous voyez à gauche **est** une maquette haute fidélité (*high-fidelity mockup*) : une représentation qui reproduit fidèlement ce que verra l'utilisateur final mêmes couleurs, mêmes polices, mêmes espacements , mêmes textes. ils est differents des maquettes basse fidélité (*low-fidelity mockups*) qui sont plus schématiques, avec des blocs gris et du texte générique (*lorem ipsum*). et permet ainsi de se concentrer sur la structure et le contenu plutôt que sur l'apparence exacte.

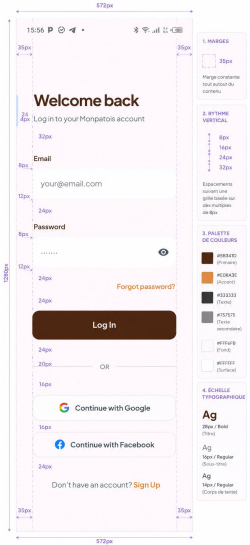
Caractéristiques principales — pointées sur notre maquette



Caractéristiques principales — synthèse

Caractéristique	**Ce qu'on observe sur notre maquette**
Précision visuelle	Couleurs exactes du bouton *Log In*, typographie définitive du titre
Contenu réaliste	« your@email.com », icône Google réelle : pas de *lorem ipsum*
Espacements cohérents	Marges verticales régulières entre label et champ
Interactivité simulée	Bouton *Log In* à l'état plein, icône « œil » sur le mot de passe
Alignement pixel-perfect	Chaque champ a une position et une largeur mesurables en pixels

Les éléments métriques — de la maquette dans son ensemble



Les éléments métriques — synthèse

Ce qu'on peut mesurer sur **la maquette entière**, pas seulement sur un élément isolé :

- **Dimension du canevas** : 572×1280 px (taille totale de l'écran)
- **Marges** : distance constante entre le bord de l'écran et le contenu (ici ≈ 35 px de chaque côté)
- **Rythme vertical** : les espacements entre blocs successifs suivent une grille régulière ($\approx$ 20–30 px, multiples de 8 px)
- **Palette de couleurs** : un jeu limité de teintes réutilisées (fond, texte, bouton, accent)
- **Échelle typographique** : quelques tailles de police seulement (titre, sous-titre, corps de texte)

Pourquoi c'est important

Ces régularités (grille, palette, échelle) sont ce qui permettra plus tard de **déduire des règles de style génériques** lors de la génération de code, plutôt que de coder chaque pixel un par un.

Qu'est-ce qu'un modèle ?

Ici, **modèle** ne désigne pas un modèle d'IA, mais une **représentation structurée et abstraite** de la maquette que nous venons de voir, sous forme de graphe ou d'arbre.

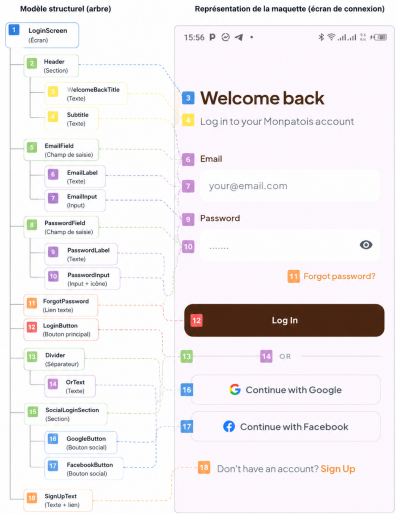
Le modèle ne contient plus de pixels : il contient

- des **éléments** (les 18 composants de notre écran de connexion)
- des **relations** entre eux (proximité, appartenance, hiérarchie)

Analogie

La maquette est une photo de famille ; le modèle est l'arbre généalogique.
La photo montre les visages, l'arbre montre qui est le parent de qui.

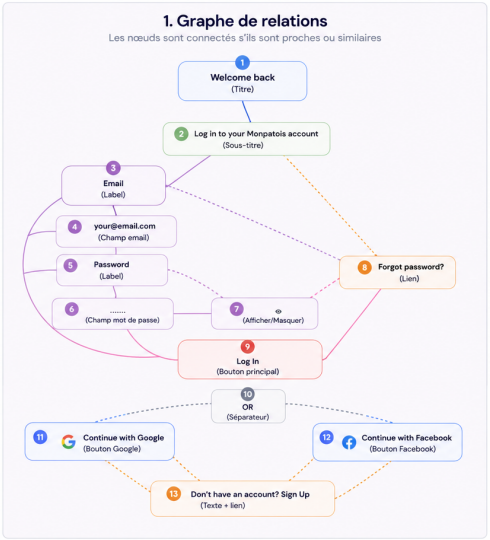
– representation

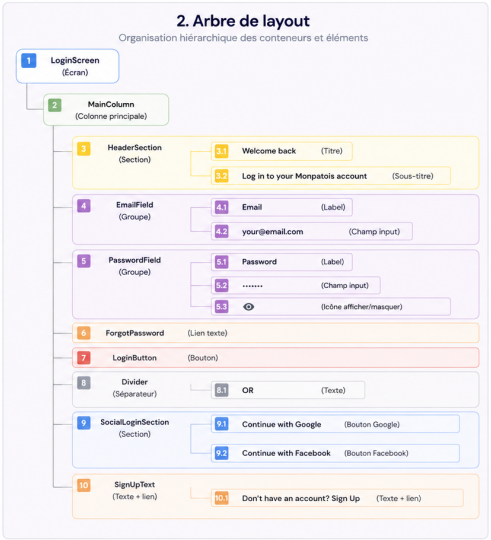


Les deux formes du modèle

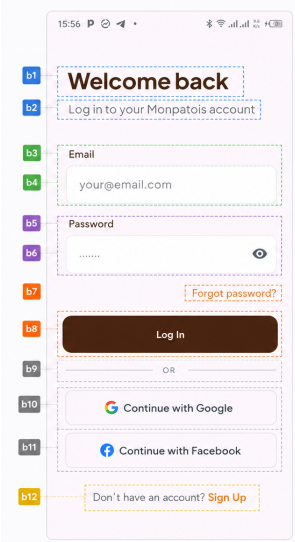
1. **Le graphe de relations**
   Chaque élément de notre maquette devient un nœud. Deux nœuds reliés s'ils sont proches ou similaires.
   → répond à : « *quels éléments vont ensemble ?* »

2. **L'arbre de layout**
   Les groupes obtenus s'organisent en hiérarchie (conteneurs, sous-conteneurs, éléments atomiques).
   → répond à : « *comment ces groupes s'emboîtent-ils ?* »


1. Graphe de relations
Les nœuds sont connectés s'ils sont proches ou similaires
1 Welcome back (Titre)
2 Log in to your Monpatois account (Sous-titre)
3 Email (Label)
4 your@email.com (Champ email)
5 Password (Label)
6 ..... (Champ mot de passe)
7 (Afficher/Masquer)
8 Forgot password? (Lien)
9 Log In (Bouton principal)
10 OR (Séparateur)
11 Continue with Google (Bouton Google)
12 Continue with Facebook (Bouton Facebook)
13 Don't have an account? Sign Up (Texte + lien)


2. Arbre de layout
Organisation hiérarchique des conteneurs et éléments
1 LoginScreen (Écran)
2 MainColumn (Colonne principale)
3 HeaderSection (Section)
3.1 Welcome back (Titre)
3.2 Log in to your Monpatois account (Sous-titre)
4 EmailField (Groupe)
4.1 Email (Label)
4.2 your@email.com (Champ input)
5 PasswordField (Groupe)
5.1 Password (Label)
5.2 ..... (Champ input)
5.3 (Icône afficher/masquer)
6 ForgotPassword (Lien texte)
7 LoginButton (Bouton)
8 Divider (Séparateur)
8.1 OR (Texte)
9 SocialLoginSection (Section)
9.1 Continue with Google (Bouton Google)
9.2 Continue with Facebook (Bouton Facebook)
10 SignUpText (Texte + lien)
10.1 Don't have an account? Sign Up (Texte + lien)

Étape 1 — Détection des boîtes sur notre maquette



Étape 1 — Le tableau des 12 boîtes détectées

ID	Élément	Type	Groupe
b_1	Welcome back	Titre	g_1
b_2	Log in to your...	Sous-titre	g_1
b_3	Email	Label	g_2
b_4	your@email.com	Champ	g_2
b_5	Password	Label	g_3
b_6	•••••••	Champ	g_3
b_7	Forgot password?	Lien	g_4
b_8	Log In	Bouton	g_4
b_9	OR	Séparateur	g_5
b_{10}	Continue with Google	Bouton	g_5
b_{11}	Continue with Facebook	Bouton	g_5
b_{12}	Sign Up	Lien	g_6

Chaque élément détecté devient un **nœud** du futur graphe, avec sa position (x,y) mesurée sur l'image.

La couleur de chaque boîte, sur la vue précédente, correspond à son **type** — pas encore à son **groupe** : c'est justement ce que le graphe va nous permettre de déterminer.

Comment décide-t-on qu'un lien existe ?

Pour construire le graphe, on ne relie pas les éléments au hasard : on calcule un **score de similarité** σ pour *chaque paire* d'éléments détectés sur notre maquette.

$$\sigma(b_i, b_j) = \alpha \cdot \mathrm{sim}_{\mathrm{geo}}(b_i, b_j) + \beta \cdot \mathrm{sim}_{\mathrm{sem}}(b_i, b_j)$$

avec $\alpha + \beta = 1$

Deux critères combinés

- $\mathrm{sim}_{\mathrm{geo}}$: sont-ils **proches sur l'écran** ?
- $\mathrm{sim}_{\mathrm{sem}}$: se **ressemblent-ils visuellement** (même style, même type) ?

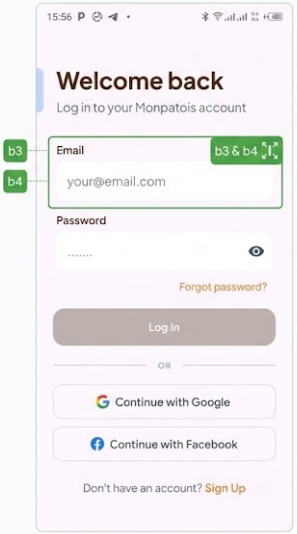
Le score σ est donc un compromis entre *position* et *apparence*.

Attention : similarite geometrique et similarite sémantique sont deux choses différentes

À ne pas confondre

- sim_{geo} mesure une **distance sur l'écran** (en pixels).
- sim_{sem} mesure une **ressemblance visuelle** (couleur, forme, type d'élément) — *indépendamment* de la position.

Deux éléments peuvent être proches sans se ressembler, ou se ressembler sans être proches.

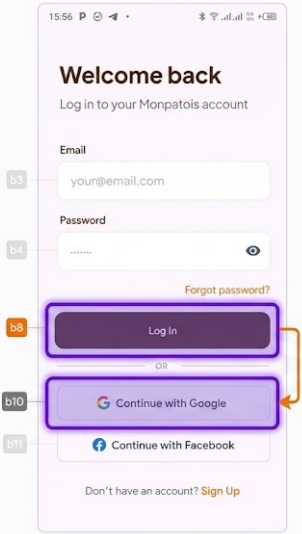


Exemple géométrique (vert)

Proximité Spatiale Maximale

- b3 (Email) et b4 (son champ) sont très proches sur l'écran.
- sim_geo(b3, b4) élevé, malgré des types di▒érents.



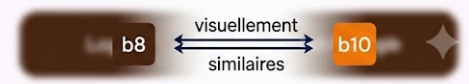


sim_sem élevé (même type)

Exemple sémantique (orange)

Regroupement par ressemblance visuelle

- **Éléments concernés :** b8 (Log In) et b10 (Continue with Google)
- **Contexte :** Séparés par OR, donc éloignés.
- **Justification :** sim_sem(b8, b10) élevé car ce sont des boutons (même forme arrondie, texte centré).
- **Conclusion :** La ressemblance de style prévaut sur la distance pour suggérer un lien.



Le détail des deux calculs

1. Similarité géométrique — distance entre les coordonnées (x, y) des deux boîtes (ex. b_3 et b_4 vus à l'instant), transformée en score entre 0 et 1 :

$$\text{sim}_{\text{geo}}(b_i, b_j) = e^{-\,\text{distance}(b_i, b_j)/\tau}$$

Plus la distance est petite, plus le score se rapproche de 1. τ règle la sensibilité à la distance.

2. Similarité sémantique — chaque élément est résumé par un vecteur de caractéristiques visuelles $\phi(b_i)$ (couleur, forme, texte ou non). On compare deux vecteurs par cosinus :

$$\text{sim}_{\text{sem}}(b_i, b_j) = \cos\big(\phi(b_i), \phi(b_j)\big)$$

Deux éléments de même nature (les boutons b_8 et b_{10} vus à l'instant, par exemple) obtiennent un score proche de 1, **même si** leur sim_{geo} est plus faible.

Lien fort ou lien faible : le rôle du seuil θ

Une fois σ calculé pour une paire, on compare ce score à un **seuil** θ fixé à l'avance (ex. $\theta = 0{,}5$) :

Règle de décision

$$\sigma(b_i, b_j) \geq \theta \;\Rightarrow\; \textbf{lien fort} \text{ (arête tracée)}$$

$$\sigma(b_i, b_j) < \theta \;\Rightarrow\; \textbf{lien faible} \text{ (pas d'arête)}$$

Sur notre maquette : b_3 et b_4 franchissent le seuil, b_3 et b_9 non — comme on l'a vu sur l'image précédente. Un « lien faible » ne se dessine pas : il correspond simplement à l'**absence** d'arête entre les deux nœuds.

Le seuil θ est un paramètre qu'on choisit : plus il est bas, plus on autorise de liens (groupes plus larges) ; plus il est haut, plus on est strict (groupes plus petits et précis).

Calcul concret — b_3 et b_4 vs. b_3 et b_9

On compare b_3 (Email) et b_4 (champ) : très proches verticalement.

$$\text{sim}_{\text{geo}}(b_3, b_4) = e^{-0.04/0.1} \approx 0.67 \qquad \text{sim}_{\text{sem}}(b_3, b_4) \approx 0.3$$

$$\sigma(b_3, b_4) = 0.6 \times 0.67 + 0.4 \times 0.3 = 0.522 \ \ (> \theta = 0.5) \ \Rightarrow \ \textbf{arête tracée}$$

On compare maintenant b_3 (Email) et b_9 (OR) : très éloignés sur l'écran.

$$\text{sim}_{\text{geo}}(b_3, b_9) = e^{-0.40/0.1} \approx 0.018 \qquad \sigma(b_3, b_9) \approx 0.051 \ \ (< \theta) \ \Rightarrow \ \textbf{pas d'arête}$$

Principe

On répète ce calcul pour **toutes les paires** des 12 boîtes détectées sur notre maquette. Seules les paires dont $\sigma \geq \theta$ sont reliées dans le graphe.

Étape 3 — Que veut dire $G = (B, E)$?

Avant de regarder le graphe, précisons la notation.

Un graphe est une paire $G = (B, E)$

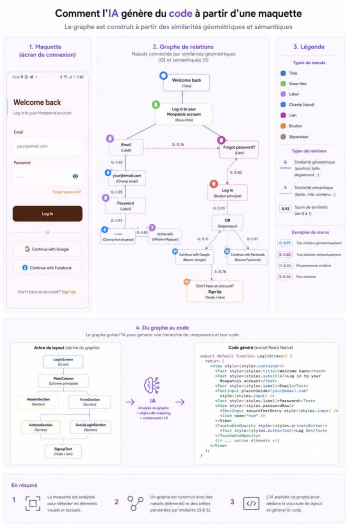
- B (*boxes*) : l'ensemble des **nœuds**, ici nos 12 boîtes détectées $b_1, \ldots, b_{12}$.
- E (*edges*) : l'ensemble des **arêtes**, c'est-à-dire les paires (b_i, b_j) telles que $\sigma(b_i, b_j) \geq \theta$.

Concrètement : si $\sigma(b_i, b_j) < \theta$ (lien faible), **on ne trace rien** — ce n'est pas une arête « en pointillé », c'est simplement l'absence de lien entre b_i et b_j.

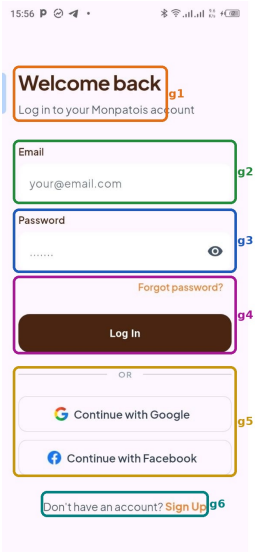
Ce qu'on va voir sur le graphe

Chaque couleur = un **groupe** g_i, c'est-à-dire une **composante connexe** : un ensemble de nœuds tous reliés (directement ou indirectement) par des arêtes de E.

Étape 3 — Le graphe obtenu à partir de notre maquette



Étape 4 — Les groupes reportés sur notre maquette



Étape 4 — Des groupes vers le modèle

Ce qu'on obtient

Le graphe s'est transformé en **6 groupes logiques**, directement visibles sur notre maquette :

- $g_1 = \{\text{Welcome back, sous-titre}\}$ — en-tête
- $g_2 = \{\text{Email, champ email}\}$ — champ formulaire
- $g_3 = \{\text{Password, champ password}\}$ — champ formulaire
- $g_4 = \{\text{Forgot password ?, Log In}\}$ — actions
- $g_5 = \{\text{OR, Google, Facebook}\}$ — connexion sociale
- $g_6 = \{\text{Sign Up}\}$ — pied de page

L'idée centrale

En une phrase

On transforme notre maquette (image, données visuelles) en modèle (graphe puis arbre, données structurelles) afin de regrouper logiquement les éléments avant de générer du code.

La transformation n'ajoute pas d'information nouvelle : elle rend explicite une structure qui existait déjà implicitement dans notre écran de connexion.

Formulation simple

La maquette montre *ce qui est affiché*.
Le modèle explique *comment c'est organisé*.

Comment les deux collaborent

Étape	Rôle de la maquette	Rôle du modèle
Avant transformation	Fournit les éléments bruts (pixels, positions, couleurs)	N'existe pas encore
Pendant la transformation	Sert de référence pour extraire les mesures	Se construit progressivement (graphe, groupes, arbre)
Après transformation	Reste disponible comme vérité visuelle de contrôle	Guide la génération de code

Références

Cette note de synthèse s'appuie sur les travaux suivants sur la reconstruction de structure à partir d'images d'interfaces :

- T. A. Nguyen et C. Csallner. *Reverse Engineering Mobile Application User Interfaces with REMAUI*. ASE 2015, p. 248–259.
- C. Chen, T. Su, G. Meng, Z. Xing et Y. Liu. *From UI Design Image to GUI Skeleton : A Neural Machine Translator to Bootstrap Mobile GUI Implementation*. ICSE 2018, p. 665–676.
- M. Xie, Z. Xing, S. Feng, X. Xu, L. Zhu et C. Chen. *Psychologically-Inspired, Unsupervised Inference of Perceptual Groups of GUI Widgets from GUI Images*. ESEC/FSE 2022.
- T. Beltramelli. *pix2code : Generating Code from a Graphical User Interface Screenshot*. ACM SIGCHI EICS 2018.

Ces travaux formalisent, chacun à leur façon, l'idée centrale de cette présentation : détecter des éléments d'interface par proximité et par similarité visuelle, puis les regrouper en une structure exploitable pour la génération de code.

Merci